You will find the collections module to be more of a practical use, wherein data structures are tweaked for easier and quicker implementation. We will be talking about some of these container data types, their usages, and implementation.

Counter

A Counter is a subclass of dict or a Dictionary in common terms, wherein we store the object and their respective counts in an unordered collection. Simply, a counter is used to count the occurrences of a hashable object. It is similar to multisets in other languages, also referred to as a bag.

A counter can be initialized in one of the following ways

- An empty `Counter`

```
>>> counter = Counter()
>>> counter
Counter()
#Represents a counter with no values.
```

- Using an iterable or a sequence of items.

```
>>> counter = Counter(['B','B','A','B','C','A','B','B
','A','C'])
>>> counter
Counter({'B': 5, 'A': 3, 'C': 2})
#Represents 5 occurrences of 'B', 3 occur-
rences of 'A' and so on.
```

- Using a `Dictionary` of predefined keys and their respective counts.

```
>>> counter = Counter({'book':4,'pen':2,'pencil': 8})
>>> counter
Counter({'pencil': 8, 'book': 4, 'pen': 2})
#The order of objects is completely randomized,
hence the insertion order is never guaranteed
```

- Using Keywords Arguments

```
>>> counter = Counter(book=4, pen=8, pencil=8)
>>> counter
Counter({'pen': 8, 'pencil': 8, 'book': 4})
```

- One can also add more data to a `Counter` post-initialization using the update method, which operates both on positive as well as negative values. Giving a negative value during an update call will lead to deletion of data from the `Counter` instead of addition.

  Let's use the following Counter to illustrate: